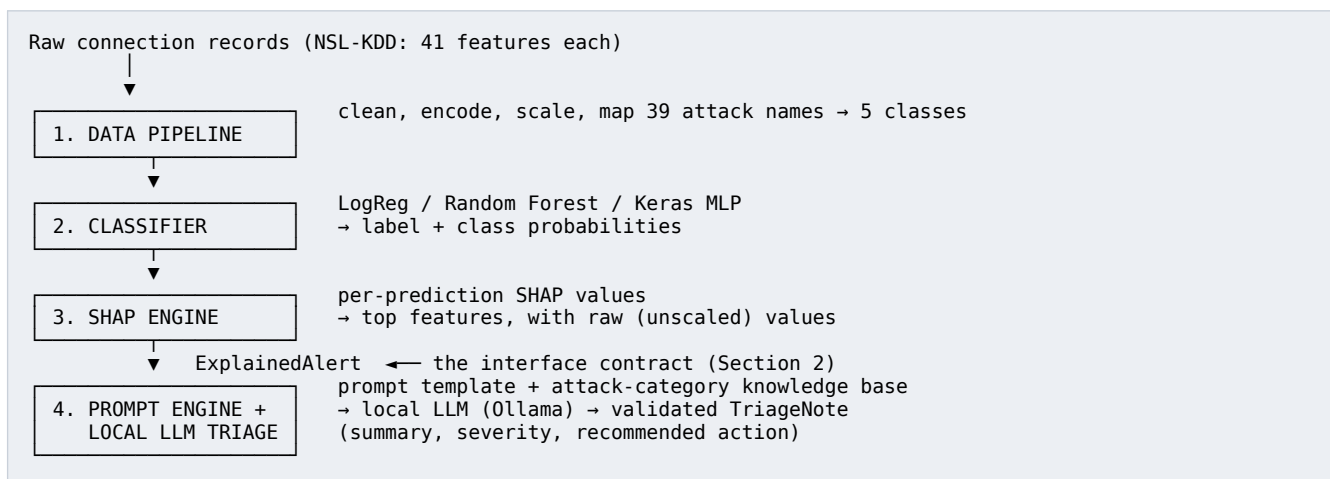


Project Plan: Explainable NIDS with LLM-Augmented Triage

Two-person, 4-week build · Local-LLM edition (free, no paid API)

What changed from the earlier version. The proposal used the Anthropic API for the triage layer. This edition replaces it with an open-weights model running on your own machine through **Ollama**, so there is no cost, no rate limit and no API key. The pipeline, the interface contract and the “LLM is the narrator, not the detector” principle are unchanged. Only Member B's LLM module changes, plus a few Week 1 setup tasks and some extra care around hallucination, because small local models are less reliable than frontier hosted ones. Section 4 covers the local setup in detail.

1. System Architecture & Conceptual Walkthrough



Stage 1: Data pipeline

In a SOC, raw telemetry is messy and inconsistent, and a model can't use it until it is in a uniform numeric shape. NSL-KDD gives you 41 features per connection plus a label. Three things need real care:

- `protocol_type`, `service` and `flag` are categorical and need encoding.
- The ~39 specific attack names must be mapped to the 4 families (DoS, Probe, R2L, U2R) plus normal.
- Scalers must be fit on training data only. Fitting on everything leaks test information and inflates your results.

Stage 2: Multi-class classifier

A SOC sees far more traffic than humans can read, so the classifier decides what deserves attention. Each baseline teaches something different:

- Logistic regression is the linear, interpretable baseline.
- Random forest is typically very strong on tabular data.
- The Keras MLP connects to your coursework.

Judge by per-class recall and macro-F1, not accuracy. In intrusion detection a missed attack is far costlier than a false alarm, and accuracy hides failures on rare classes.

Stage 3: SHAP engine

Analysts won't act on "ATTACK: 0.93" with no reason, and you can't tell whether the model learned something real or a quirk of the data. The intuition behind SHAP is that it divides credit for a prediction among the features. Starting from the model's average output, it tells you how much each feature pushed this particular prediction up or down. The output is also what makes Stage 4 possible, because it is structured evidence rather than a black-box score.

Stage 4: Prompt engine and local LLM triage

Analysts spend much of their time turning detections into readable tickets and deciding priority. The local LLM does that translation. The design principle to hold onto is that **the LLM is the narrator, not the detector**. It never decides whether something is an attack. It explains a decision the classifier already made, using only evidence you hand it. The "retrieval step" in your proposal can just be a Python dict mapping the predicted class to a reference paragraph (and a glossary of the cryptic feature names). Vector search isn't needed. This matters even more with a small local model: the less it has to know on its own, the less it can get wrong.

2. Roles & Interface Contract

Member A (Data + Models + SHAP): preprocessing, all three models, evaluation, SHAP extraction, and `explain()`.

Member B (LLM layer + Benchmark): Ollama setup and model choice, knowledge base, prompts, Pydantic schemas, hallucination checks, and the Week 4 zero-shot benchmark.

Evening out the load. B is light in Weeks 1 to 2, so B could take the logistic regression baseline and the EDA correlation plots once A's preprocessed data exists (around day 3). B also needs to understand the data deeply for the zero-shot benchmark. Since you both want to understand everything, run a short teach-back each week where each of you explains your component to the other.

The contract (`contract.py`, the single source of truth)

```
from pydantic import BaseModel
from typing import Literal

AttackClass = Literal["normal", "DoS", "Probe", "R2L", "U2R"]

class FeatureContribution(BaseModel):
    feature: str          # e.g. "count"
    value: float | str     # RAW human-readable value (count=511), NOT the scaled one
    shap_value: float      # signed contribution toward the predicted class

class ExplainedAlert(BaseModel):          # A -> B
    alert_id: str
    prediction: AttackClass
    confidence: float                     # probability of predicted class
    class_probabilities: dict[str, float]
    top_features: list[FeatureContribution] # top 5, sorted by |shap_value|
    raw_record: dict[str, float | str]     # all 41 original features (for B's benchmark)
    true_label: AttackClass | None         # evaluation only; NEVER put in a prompt
    model_name: str

class TriageNote(BaseModel):              # B -> final output
    summary: str
    severity: Literal["low", "medium", "high", "critical"]
    recommended_action: str
    evidence_cited: list[str]              # must be a subset of top_features names
    uncertainty_note: str | None
```

Three details matter here:

- value must be the unscaled number. The LLM can reason about “511 connections to the same host” but not about “z-score 3.2”.
- SHAP values must be for the predicted class only.
- true_label is for evaluation, and leaking it into a prompt would invalidate your results.

Unblocking trick. On day 1 to 2, A writes a stub `explain()` that returns a few hand-written `ExplainedAlert` objects (one per class, plus a low-confidence one and a false-positive one). B builds against the stub immediately, and the real function replaces it later with no code changes on B's side. A should also freeze `data/eval_holdout.csv` (the held-out set for the Week 4 benchmark) by end of Week 1, so B's experiment isn't blocked.

3. Synchronized 4-Week Schedule

	Member A	Member B
Week 1	• Load/clean NSL-KDD, encode, scale (fit on train only), label mapping, EDA on class balance. • Train RF + Keras MLP; evaluate per class with a confusion matrix. • Freeze the holdout.	• Set up repo. Install Ollama, pull a model, confirm it runs and returns schema-valid JSON. • Pydantic basics. • Write the knowledge base (5 class descriptions + 41-feature glossary). • Draft prompt v1 and run it on stub alerts. • Optionally take the LogReg baseline once data is ready.
Week 2	• SHAP on the chosen model. • Sanity check against known signatures (e.g., DoS ↔ count / srv_count / error_rate). • Implement the real <code>explain()</code>. • Generate 50 real alerts as JSON.	• Forced-schema output via Ollama's format option, with validation and one retry. • Automatic checker: cited features and numbers actually appear in the input alert. • Add a severity rubric to the prompt. • Log every prompt/response. • Compare 2 candidate local models on the stub alerts; pin one. • Build edge-case alerts (low confidence, benign false positive).
Week 3	• Select the 30-50 review samples (stratified, plus misclassified and low-confidence ones). • Build the manual review sheet and rubric. • Cache SHAP results. • Start error analysis and the demo notebook.	• Batch-run triage on the real alerts. • Iterate the prompt based on review findings. • Track latency and failure rate.
Week 4	• Demo notebook / Streamlit (only if time allows). • Model-side results tables and limitations.	• Zero-shot / few-shot benchmark on the frozen holdout vs. the trained model. • Write the benchmark and LLM sections of the README, naming the exact local model and settings.

Integration checkpoints

- 1. End of Day 2:** contract frozen and stub merged. Changes after this require both of you to agree.
- 2. End of Week 1:** the preprocessed data, the frozen holdout, and the chosen model are handed over.
- 3. End of Week 2:** the real `explain()` replaces the stub and the 50-alert JSON is delivered. Pair-program the first end-to-end run (sample → verdict → SHAP → note).
- 4. Mid Week 3:** both of you blind-review the same 15 outputs, then compare ratings to calibrate your hallucination rubric.
- 5. Start of Week 4:** freeze the prompts, models and Ollama settings, so benchmark numbers are all computed against one fixed version.

4. Running the LLM Locally with Ollama (Free)

Ollama is a free tool that downloads open-weights language models and serves them on your own machine. Nothing is sent to a third party, which also removes any concern about where your data goes. Your triage module talks to it from Python.

Setup (about 15 minutes per person)

1. Install Ollama from ollama.com for your OS, then check it works with `ollama --version`.
2. Pull a model, e.g. `ollama pull llama3.1:8b` or `ollama pull qwen2.5:7b`. Check ollama.com/library for current options; newer models may be better.
3. Quick test in a terminal: `ollama run llama3.1:8b`, type a prompt, then exit.
4. Install the Python client: `pip install ollama`.
5. **Pin one exact model tag** in a shared config file so both of you generate results from the same model.

Choosing a model (rough guidance)

Model size	Approx. memory (4-bit)	Trade-off
~3B (e.g. llama3.2:3b)	~2–3 GB	Runs on almost any laptop; weakest at following the schema and staying grounded.
~7–8B (e.g. llama3.1:8b, qwen2.5:7b)	~5–6 GB	Best default for this project. Usable on a 16 GB laptop; much faster with a GPU.
~14B and larger	~9 GB or more	More reliable but slow without a good GPU. Try only if your hardware allows.

These figures are approximate and vary by model and settings. If a machine is too weak, CPU-only inference still works but is slow (tens of seconds per note), which is acceptable for 30–50 samples but painful for prompt iteration. A free Colab GPU can also host Ollama, but sessions reset, so you would need to reinstall and re-pull the model each time.

The one function Member B writes against

Keep the provider behind a single function so the rest of the pipeline never knows which LLM is underneath. Passing the Pydantic JSON schema as `format` makes Ollama constrain its output to that schema; you still validate afterwards. (Check the current [ollama-python docs](#) if the response object differs in your version.)

```

import ollama
from contract import TriageNote

MODEL = "llama3.1:8b"      # pin ONE tag in shared config; both members use it

def call_llm(system_prompt: str, user_prompt: str) -> TriageNote:
    last_error = None
    for attempt in range(2):          # first try + one retry
        resp = ollama.chat(
            model=MODEL,
            messages=[{"role": "system", "content": system_prompt},
                     {"role": "user", "content": user_prompt}],
            format=TriageNote.model_json_schema(), # constrain output to the schema
            options={"temperature": 0, "seed": 42}, # as deterministic as possible
        )
        try:
            return TriageNote.model_validate_json(resp.message.content)
        except Exception as e:        # invalid JSON or schema violation
            last_error = e
    raise ValueError(f"LLM output failed validation twice: {last_error}")

```

What local models change in practice

- **More hallucination risk.** Small models invent details more readily, so the automatic checker (cited features and numbers must appear in the input alert) is not optional. Keep prompts short and the evidence block tightly structured.
- **Schema-valid does not mean correct.** Constrained output guarantees the shape, not the truth. Your evidence-subset check and manual review are what catch fabricated content.
- **Reproducibility.** Fix temperature, seed and model tag, and log them with every run. Different hardware can still give slightly different text, so save all raw outputs rather than regenerating.
- **Speed.** Triage only flagged alerts, cache every response keyed by (alert_id, prompt version, model), and never re-run the whole batch for a one-line prompt change when a 5-sample test will do.
- **Benchmark expectations.** A small local model will likely do worse zero-shot than a large hosted one. That is a legitimate finding, not a failure: report the model name and size and say the result applies to it.

5. Failure Modes & Early Mitigation

1. Class imbalance and distribution shift

The training set has roughly 67k normal, 46k DoS, 12k Probe, about 1k R2L, and only 52 U2R records. KDDTest+ also contains attack types not in training. Expect weak R2L and U2R recall, and a notable accuracy drop on the test set versus validation.

- Use the official KDDTrain+/KDDTest+ split rather than a random split, and carve validation out of train.
- Use `class_weight="balanced"`, and report macro-F1 and per-class recall instead of accuracy.
- If you try SMOTE, apply it after splitting and to training data only.
- Treat poor U2R performance as a finding to report, not a bug to hide.

2. SHAP friction

There are three separate issues:

- The output shape for multi-class models differs across SHAP versions (a list of arrays vs. one 3D array), so print shapes and write a small adapter on day 1.

- TreeExplainer on Random Forest is fast, but Keras explanations (DeepExplainer/GradientExplainer, or KernelExplainer with a small k-means background) are slower and fussier. Unless the MLP clearly wins, explain the Random Forest. Either way, only compute SHAP for flagged samples and cache the results.
- One-hot columns like `service_http` fragment credit, so map them back to the original feature name. Correlated features (e.g., `error_rate` and `srv_error_rate`) also split credit, so top-5 lists may look redundant. SHAP shows what the model used, not what's causally true.

3. LLM hallucination and format drift (local model)

- Force structured output with Ollama's `format` option and your Pydantic-derived JSON schema, validate with Pydantic, and retry once on failure.
- Set temperature to 0 and a fixed seed, and instruct the model to use only the provided evidence and to say so when evidence is thin.
- Add a programmatic check that `evidence_cited ⊆ top_features`, and that any quoted numbers match the input.
- Make severity rules explicit in the prompt (e.g., based on class and confidence). There is no ground truth for severity, so be honest in the README that it is rated against your own rubric.
- Iterate prompts on a smaller model for speed, then confirm on the pinned final model. If outputs stay unreliable, move up one model size before adding prompt complexity.
- Keep prompts short. Small models lose track of long instructions, so give one clear rubric and a single example.

4. Coordination drift

Mismatched schemas and notebook merge conflicts can silently break the pipeline.

- Keep shared code in `.py` modules (notebooks for exploration only).
- Strip notebook outputs with `nbstripout`.
- Fix random seeds, and keep the pinned Ollama model tag and options in one shared config file.
- Make contract changes a pull request both of you approve.

A caveat for the Week 4 benchmark

NSL-KDD is a well-known public dataset and may appear in an LLM's training data. Mention contamination as a limitation, and be careful to give the LLM and the trained model the exact same held-out records.